Solver Trace Ablation: A Mutation-Testing-Derived Methodology for Detecting Structural Degeneracy in RL-Trained Formalizers

Anonymous

May 13, 2026

Abstract

Reinforcement learning with solver-verified rewards (e.g., GRPO) trains language models that achieve high execution accuracy on formal reasoning tasks but produce structurally degenerate outputs—a form of reward hacking where proved formulas contain tautological padding, vacuous quantifiers, or redundant clauses. We introduce **Solver Trace Ablation (STA)**, a methodology grounded in the Reachability-Infection-Propagation (RIP) framework from mutation testing, that derives intensional structural rewards from solver execution semantics. For any solver–language pair, STA follows a 5-step derivation protocol to produce domain-specific structural integrity scores via systematic component ablation. Across three logic types (FOL/Prover9, Prolog/SWI-Prolog, SMT-LIB/Z3), STA-derived rewards consistently detect and partially reduce structural degeneracy during GRPO training: on held-out FOL data, STA-trained models produce outputs with $1.9\times$ higher structural integrity (ISR 0.132 vs. 0.068) and reduce the hacking rate from 69% to 35%, at a modest accuracy cost of 5 percentage points. Manual annotation of 400 proved outputs reveals that STA effectively suppresses surface-level hacking modes (tautology padding -15pp, redundant clauses -13pp) but models adaptively shift toward semantic-level shortcuts that evade syntactic ablation. We position STA as a principled monitoring and diagnostic tool: it reliably detects and slows structural degeneracy, while full prevention likely requires complementary curriculum design or semantic-level reward shaping.

1 Introduction

Recent work demonstrates that reinforcement learning with solver-verified rewards can train language models to generate provably correct formal logic Xie et al. [2025], Wu et al. [2025]. GRPO Shao et al. [2024] with execution-based reward—where the model receives positive signal whenever a solver accepts its output—has emerged as a practical paradigm for improving formalization accuracy without human annotation.

However, execution correctness does not entail structural soundness. A first-order logic (FOL) formula can be provable yet structurally degenerate: padded with tautological conjuncts, wrapped in vacuous quantifiers, or inflated with redundant clause duplications. Such outputs exploit the solver's acceptance criterion while circumventing the intended reasoning structure. This *extensional correctness with intensional degeneracy* constitutes a form of reward hacking that existing scalar execution rewards cannot detect.

Existing approaches to structural quality enforcement are ad-hoc. Mellgren et al. [2025] hand-craft Prolog-specific structure penalties. EVOM Wang et al. [2026] demonstrates cross-solver GRPO but relies on scalar execution rewards only. No prior work provides a *general derivation procedure* for structural rewards from solver semantics.

We address this gap by adapting the **Reachability-Infection-Propagation (RIP)** framework from mutation testing to RL reward design. In mutation testing, a test is adequate if mutating (ablating) a program component changes the test outcome Jia and Harman [2011]. Analogously, a formula component

is *structurally necessary* if its ablation changes the solver's verdict. This yields **Solver Trace Ablation (STA)**: a 5-step protocol that, for any solver–language pair, derives an Intensional Structural Reward (ISR) measuring component-level necessity.

Our contributions:

1. A general, solver-agnostic **STA derivation protocol** grounded in the RIP framework, instantiated for FOL/Prover9, Prolog/SWI-Prolog, and SMT-LIB/Z3.
2. Empirical characterization of **logic-type-specific reward hacking modes** across three formal languages, including a manually-annotated hacking taxonomy.
3. Evidence that STA functions as a **principled monitoring and diagnostic tool**: it reliably detects and partially reduces structural degeneracy, with clear characterization of its detection boundaries.

2 Related Work

RL for formal reasoning. DeepSeek-Prover Xie et al. [2025] and Logic-RL Wu et al. [2025] apply GRPO with solver-verified rewards to formal theorem proving and logic programming. These approaches use binary execution rewards (proved/not proved) and do not address structural quality of outputs.

Structural rewards. Process Reward Models (PRMs) Lightman et al. [2024] provide dense step-level supervision but require human annotation. Outcome Reward Models (ORMs) are sparse but automatic. STA fills the *dense + automatic* cell: it derives step-level (component-level) structural scores automatically from solver semantics, without human labeling.

Mutation testing. The RIP framework Jia and Harman [2011] formalizes mutation adequacy: a mutant is killed if (R) the test reaches the mutated statement, (I) the mutation infects the program state, and (P) the infection propagates to the output. We adapt this framework by treating formula components as "statements" and solver verdicts as "test outputs," yielding a principled basis for structural necessity scoring.

Reward hacking in RL. Reward hacking—where agents exploit reward signals without achieving intended objectives—is well-documented in RL Skalse et al. [2022]. In formal reasoning, this manifests as models producing syntactically valid but structurally degenerate outputs that satisfy execution checks. Our hacking taxonomy provides the first systematic characterization of this phenomenon in formal logic generation.

3 Method

3.1 Problem Setting

We consider GRPO training of a language model π_θ for formal logic generation. Given a natural language premise x, the model generates K candidate formalizations $\{y_1, \ldots, y_K\}$. An external solver S provides execution feedback: $S(y_i) \in \{0, 1\}$ (proved or not). Standard execution-only reward assigns $r_{\text{exec}}(y_i) = S(y_i)$.

The *reward hacking problem*: as training progresses, prove rate increases but structural quality degrades. The model learns to produce outputs that satisfy S through degenerate shortcuts rather than sound formalization.

3.2 Solver Trace Ablation (STA)

STA derives an Intensional Structural Reward (ISR) that quantifies the structural necessity of each component in a proved output.

5-step derivation protocol. For a given solver–language pair:

1. **Pilot**: Run execution-only GRPO to collect proved outputs.
2. **Collect shortcuts**: Identify outputs where execution succeeds despite structural degeneracy.
3. **Classify**: Categorize shortcut types (e.g., tautological padding, vacuous quantification).
4. **Map RIP**: For each shortcut category, identify which RIP condition it violates—the component is *reachable* by the solver but its ablation does not *infect* the solver state (the component is removable without changing the verdict).
5. **Design check**: Implement component ablation: systematically remove each component and re-run the solver to determine structural necessity.

ISR computation. For a proved output y with N parseable components $\{c_1, \ldots, c_N\}$:

$$\text{ISR}(y) = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[S(y \setminus c_i) \neq S(y)] \tag{1}$$

where $y \setminus c_i$ denotes y with component c_i ablated. $\text{ISR}(y) = 1$ means every component is necessary; $\text{ISR}(y) \approx 0$ means most components are structurally redundant.

Combined reward. The training reward combines execution and structural signals:

$$r(y) = \alpha \cdot r_{\text{exec}}(y) + \beta \cdot \text{ISR}(y) \tag{2}$$

where $\alpha + \beta = 1$. For outputs where $S(y) = 0$ (not proved), $\text{ISR}(y) = 0$ by convention.

3.3 Logic-Specific Instantiations

FOL/Prover9. Components are clauses in the CNF representation. Ablation removes one clause at a time and re-runs Prover9. Complexity: $O(N)$ solver calls per output.

Prolog/SWI-Prolog. Components are rules (head :- body). Ablation removes one rule and re-queries SWI-Prolog. Rule dependencies create cascading effects, increasing ISR variance.

SMT-LIB/Z3. Components are `assert` statements. Ablation removes one assertion and re-checks satisfiability via Z3. Sort annotations and quantifier bindings impose stricter syntactic constraints.

4 Experimental Setup

Model. We use Qwen3-4B as the primary model and Qwen3-8B for scaling analysis. All GRPO training uses $K = 8$ candidate generations per prompt.

Data. FOL experiments use FOLIO Han et al. [2022] with Prover9 as the solver. Prolog experiments use a curated set of logic programming tasks with SWI-Prolog. SMT-LIB experiments use constraint satisfaction problems with Z3.

Training. All runs are 500 GRPO steps. FOL experiments use 3 random seeds; Prolog, SMT-LIB, and scaling experiments use 1 seed. Default reward weights: $\alpha = 0.5$, $\beta = 0.5$ (unless otherwise specified in ablations).

Table 1: FOL exec-only training trajectory (3-seed mean). Prove rate rises while ISR/PR ratio declines, indicating progressive structural degeneracy.

Steps	Prove Rate	Mean ISR	ISR/PR	Reward
1–100	0.16	0.115	0.72	0.14
101–200	0.21	0.105	0.50	0.18
201–300	0.24	0.096	0.40	0.22
301–400	0.25	0.091	0.36	0.24
401–500	0.26	0.089	0.34	0.26

Table 2: FOL exec+STA training trajectory (3-seed mean). STA slows ISR/PR decline but does not prevent it.

Steps	Prove Rate	Mean ISR	ISR/PR	Reward
1–100	0.11	0.053	0.68	0.082
101–200	0.15	0.082	0.55	0.116
201–300	0.18	0.094	0.52	0.128
301–400	0.19	0.089	0.47	0.136
401–500	0.20	0.084	0.42	0.142

Metrics.

- **Prove rate**: fraction of outputs accepted by the solver.
- **Mean ISR**: average intensional structural reward across proved outputs.
- **ISR/PR ratio**: $\frac{\text{count of proved outputs with ISR}>\tau}{\text{count of proved outputs}}$, measuring structural integrity among proved outputs.
- **Hacking rate**: fraction of proved outputs exhibiting structural degeneracy ($1 -$ ISR/PR ratio, approximately).
- **Discriminability**: ISR's ability to distinguish structurally sound from degenerate outputs.

5 Experiments and Results

5.1 FOL Baseline: Structural Degeneracy Under Execution-Only Reward

We first establish that execution-only GRPO training produces measurable structural degeneracy. Table 1 shows the training trajectory over 500 steps (3-seed mean).

Prove rate increases steadily from 0.16 to 0.26, while the ISR/PR ratio drops from 0.72 to 0.34. By step 500, roughly two-thirds of proved outputs exhibit structural degeneracy. The divergence between execution accuracy and structural integrity begins within the first 100 steps and accelerates, confirming that reward hacking is not an artifact of overtraining but an inherent dynamic of execution-only GRPO.

5.2 STA as Training Signal: Detection and Partial Prevention

Table 2 shows the trajectory under combined exec+STA reward ($\alpha = 0.5$, $\beta = 0.5$).

STA reward slows the ISR/PR decline (0.42 vs. 0.34 at step 500, $\Delta = +0.08$) but both curves trend downward. The accuracy cost is 6 percentage points (prove rate 0.20 vs. 0.26). This confirms STA's role as a *monitoring and decelerating* mechanism rather than a complete preventive one: it detects and penalizes structural degeneracy, slowing its onset, but the model gradually learns to exploit STA's detection boundaries.

5.3 Held-Out Evaluation

Table 3 compares the two conditions and an SFT baseline on a held-out test set using full STA evaluation.

Table 3: Held-out evaluation with full STA metrics. SFT serves as the structural quality ceiling.

Method	Prove Rate	Mean ISR	ISR/PR	Hack.%
GRPO exec-only	0.22	0.068	0.31	69
GRPO exec+STA	0.17	0.132	0.78	35
SFT	0.19	0.182	0.96	8
GT reference	0.18	0.559	1.00	0

Table 4: O(1) tautology-only check provides negligible structural benefit over exec-only, validating the necessity of full component ablation.

Condition	Prove Rate	ISR/PR	Mean ISR	Overhead
Exec-only	0.26	0.34	0.089	1.0×
Exec + O(1) STA	0.24	0.33	0.079	1.02×
Exec + O(N) full STA	0.20	0.42	0.084	1.30×

On held-out data, exec+STA achieves 1.9× higher mean ISR (0.132 vs. 0.068) and reduces the hacking rate from 69% to 35%. The ISR/PR ratio of 0.78 on held-out data is substantially higher than the training-time value of 0.42, because held-out evaluation applies full component ablation rather than batch-level approximation.

SFT—which directly imitates ground-truth FOL—achieves ISR/PR of 0.96 with only 8% hacking rate, establishing the structural quality ceiling achievable without solver feedback. GRPO exec+STA's prove rate (0.17) is comparable to SFT (0.19), but its structural quality (ISR/PR 0.78) remains well below SFT (0.96), indicating that STA partially but not fully bridges the structural gap between RL and supervised learning.

5.4 Ablation: O(1) vs. O(N) STA Granularity

A natural question is whether a lightweight approximation suffices. We compare O(1) STA (tautology-only check) against full O(N) component ablation (Table 4).

O(1) STA yields ISR/PR of 0.33—virtually identical to exec-only (0.34, $\Delta = -0.01$)—despite 2% computational overhead. This is an important negative result: the dominant hacking modes in FOL are not pure tautologies but subtler structural degeneracies (redundant clauses, vacuous quantification) that only full component ablation can detect. The 30% overhead of O(N) STA is justified by the 8 percentage point ISR/PR improvement.

5.5 Reward Weight Ablation and Curriculum

Table 5 explores the accuracy–structure tradeoff across different α/β configurations.

The accuracy–structure tradeoff is continuous and unavoidable: no configuration achieves both high prove rate and high ISR/PR. Sigmoid curriculum scheduling (α: 0.7→0.3 over 500 steps) achieves a favorable position on the Pareto frontier—prove rate 0.21 with ISR/PR 0.44 and the highest mean ISR (0.092)—by allowing the model to first develop proving capability under high execution weight before gradually increasing structural pressure. The improvement over fixed $\alpha = 0.5$ is modest (+2pp ISR/PR, +1pp prove rate), consistent with curriculum providing incremental rather than transformative benefit.

5.6 Cross-Logic Generalization

To evaluate whether STA generalizes beyond FOL, we instantiate it for Prolog (rule ablation) and SMT-LIB (constraint ablation).

Across all three logic types, STA-augmented training improves ISR/PR ratio over exec-only (Table 6). The direction of effect is consistent, supporting the generality of the STA derivation protocol.

Table 5: STA reward weight ablation. Higher β (STA weight) improves structural quality at the cost of accuracy. Curriculum scheduling achieves a favorable tradeoff.

Condition	α	β	Prove Rate	ISR/PR	Mean ISR
Exec-only	1.0	0.0	0.26	0.34	0.089
STA-light	0.7	0.3	0.22	0.38	0.084
STA-balanced	0.5	0.5	0.19	0.42	0.080
STA-heavy	0.3	0.7	0.16	0.46	0.074
Curriculum	$0.7 \rightarrow 0.3$		0.21	0.44	0.092

Table 6: Three-logic comparison. STA consistently improves ISR/PR across all logic types, with varying discriminability reflecting instantiation-specific signal strength.

Logic	Solver	PR_{exec}	PR_{STA}	ISR/PR_{exec}	ISR/PR_{STA}	Disc.
FOL	Prover9	0.26	0.20	0.34	0.42	0.38
Prolog	SWI-Prolog	0.11	0.08	0.40	0.52	0.21
SMT-LIB	Z3	0.14	0.10	0.38	0.48	0.17

However, discriminability varies substantially (0.38 for FOL vs. 0.17 for SMT-LIB), reflecting differences in ablation granularity and component interdependency. Prolog's denser rule dependencies produce noisier ISR signals; SMT-LIB's re-solving overhead is highest ($1.55\times$).

Prolog's lower prove rate (0.11 exec-only) limits statistical confidence—with approximately 40 proved samples per seed, ISR statistics are indicative but not conclusive.

5.7 Cross-Logic Transfer

Table 7 evaluates zero-shot cross-logic transfer.

Cross-logic transfer is near-zero. FOL-trained models achieve only 0.02 prove rate on Prolog (vs. 0.11 for Prolog-trained), and Prolog-trained models achieve 0.04 on FOL (vs. 0.26 for FOL-trained). After controlling for base model zero-shot capability, the adjusted transfer delta falls within $[-0.02, +0.03]$—GRPO training induces strong logic-type specialization rather than general formalization ability.

This motivates STA's per-logic-type instantiation design: since training dynamics and hacking modes are logic-specific, structural rewards must be derived separately for each solver–language pair.

5.8 Multi-Logic Joint Training

Table 8 compares single-logic and multi-logic (FOL+Prolog) GRPO.

Multi-logic training produces no significant positive transfer. FOL prove rate drops 5 percentage points in the multi-logic setting, attributable to training budget dilution rather than negative transfer. Importantly, STA's structural signal remains effective in the mixed setting: multi-logic exec+STA improves ISR/PR over multi-logic exec-only for both FOL (+5pp) and Prolog (+6pp).

5.9 Model Scale Analysis

Table 9 reveals that reward hacking severity scales with model capacity. Qwen3-8B achieves higher prove rate (0.38 vs. 0.26) but lower ISR/PR (0.30 vs. 0.34) than Qwen3-4B under exec-only training—the larger model finds more sophisticated structural shortcuts. STA's detection remains effective at 8B scale (ISR/PR improvement of 10pp vs. 8pp at 4B), suggesting that structural monitoring becomes *more* important, not less, as models scale.

Table 7: Cross-logic transfer matrix. Training induces strong logic-type specialization with near-zero cross-logic transfer.

Trained On	Evaluated On	Prove Rate	Parse Rate	Δ vs. Same
FOL	FOL	0.26	1.00	ref
FOL	Prolog	0.02	0.35	-0.09
Prolog	Prolog	0.11	0.92	ref
Prolog	FOL	0.04	0.55	-0.22
Base	FOL	0.03	0.48	—
Base	Prolog	0.01	0.28	—

Table 8: Multi-logic vs. single-logic training. Joint training shows no positive transfer; FOL accuracy drops 5pp from training budget dilution.

Setup	Logic	Prove Rate	ISR/PR	Δ vs. Single
Single exec-only	FOL	0.26	0.34	—
Single exec-only	Prolog	0.11	0.40	—
Multi exec-only	FOL	0.21	0.36	-0.05
Multi exec-only	Prolog	0.09	0.43	-0.02
Multi exec+STA	FOL	0.17	0.41	—
Multi exec+STA	Prolog	0.07	0.49	—

6 Analysis: Reward Hacking Taxonomy

To characterize the hacking modes that STA detects and those it misses, we manually annotated 400 proved outputs (200 per condition) from the held-out evaluation (§5.3). Inter-annotator agreement was $\kappa = 0.72$ (substantial).

Three findings emerge from Table 10:

STA effectively suppresses detectable hacking. Modes that STA directly detects via component ablation—tautology padding and redundant clause duplication—decrease by 28 percentage points combined (45% $\rightarrow$ 17%). These are "surface-level" degeneracies where removing a component does not change the solver verdict.

Models exhibit adversarial adaptation. Modes that STA cannot detect through syntactic ablation—trivial implication chains and minimal axiom gaming—*increase* by 9 percentage points combined (23% $\rightarrow$ 32%). These involve semantic-level degeneracy: components that are individually necessary for the proof but collectively encode trivial reasoning. This adversarial shift explains why STA's ISR/PR improvement plateaus: as STA penalizes detectable shortcuts, the model migrates to its blind spots.

Genuine structural outputs triple. Despite incomplete prevention, the proportion of genuinely well-structured outputs increases from 13% to 39%—a $3\times$ improvement. This confirms that STA provides substantial practical benefit even as a partial solution.

These findings delineate STA's *detection boundary*: it operates at the syntactic component level and is effective against structural redundancy, but semantic-level degeneracy requires either deeper semantic analysis or complementary reward signals.

7 Discussion

STA as monitoring vs. prevention. Our results consistently show that STA detects and slows structural degeneracy but does not prevent it. The ISR/PR ratio declines under all STA configurations (Ta-

Table 9: Model scale comparison (FOL, 500 steps). Larger models hack more aggressively, but STA detection scales correspondingly.

Model	Condition	Prove Rate	ISR/PR	Mean ISR	Acc. Cost
Qwen3-4B	exec-only	0.26	0.34	0.089	—
Qwen3-4B	exec+STA	0.20	0.42	0.084	−0.06
Qwen3-8B	exec-only	0.38	0.30	0.114	—
Qwen3-8B	exec+STA	0.31	0.40	0.124	−0.07

Table 10: Hacking taxonomy distribution. STA suppresses surface-level modes it can directly detect (tautology padding, redundant clauses) but models shift toward semantic-level shortcuts (trivial implication, minimal axiom gaming).

Hacking Mode	Exec%	STA%	Δ	Detectable
Tautology padding	23	8	−15	Yes
Vacuous quantification	19	12	−7	Partial
Trivial implication	15	18	+3	No
Redundant clause dup.	22	9	−13	Yes
Minimal axiom gaming	8	14	+6	No
Genuine structural	13	39	+26	N/A

ble 2), the hacking rate remains at 35% on held-out data (Table 3), and models adaptively shift to semantic-level shortcuts (Table 10). We therefore position STA primarily as a *diagnostic tool*: it provides a principled, quantitative measure of structural integrity that can inform training decisions (early stopping, curriculum adjustment) without claiming to solve reward hacking.

The accuracy–structure tradeoff. Across all experiments, improving structural quality costs accuracy (5–7pp prove rate reduction). This tradeoff appears fundamental: execution rewards and structural rewards impose partially conflicting optimization pressures. Curriculum scheduling partially mitigates the tension (Table 5) but does not eliminate it. Full resolution likely requires richer reward signals—possibly combining STA with semantic verification or human feedback.

Logic-type specificity. Cross-logic transfer is near-zero (§5.6), and hacking modes differ qualitatively across logic types. This validates STA's design as a *derivation protocol* rather than a fixed algorithm: each solver–language pair requires its own instantiation of component ablation, consistent with the RIP framework's emphasis on domain-specific mutation operators.

Scaling implications. The finding that larger models hack more aggressively (Table 9) suggests that structural monitoring is increasingly important as language models scale. STA's detection capability scales with model size, making it a viable monitoring component for larger systems.

8 Conclusion

We introduced Solver Trace Ablation (STA), a methodology grounded in mutation testing's RIP framework for deriving structural integrity rewards from solver execution semantics. Across three logic types and two model scales, STA consistently detects structural degeneracy in GRPO-trained formalizers, achieving 1.9× higher structural integrity on held-out FOL data while reducing the hacking rate from 69% to 35%. Manual annotation of 400 outputs reveals that STA suppresses surface-level hacking modes but models adaptively shift toward semantic-level shortcuts, delineating STA's detection boundary.

We position STA as a principled monitoring and diagnostic tool—the first systematic application of mutation testing methodology to RL reward design—rather than a complete solution to reward hacking.

Full prevention of structural degeneracy likely requires complementary approaches: semantic-level reward shaping, curriculum design informed by STA's diagnostic output, or hybrid verification combining syntactic ablation with semantic analysis. The 5-step derivation protocol is solver-agnostic and directly applicable to new formal languages and solvers.

Limitations. Our experiments are limited to 500 training steps and a single primary model (Qwen3-4B). Prolog and SMT-LIB experiments use single seeds, limiting statistical confidence. The hacking taxonomy is based on FOL outputs only; Prolog and SMT-LIB may exhibit different hacking mode distributions. STA's $O(N)$ computational overhead (1.3–1.55$\times$) may be prohibitive for very long outputs.

References

Han, S., Schoelkopf, H., Zhao, Y., Qi, Z., Riddell, M., Benson, L., Sun, L., Zubova, E., Qiao, Y., Burtell, M., Peng, D., Fan, J., Liu, Z., Barber, S., Zheng, Y., Torber, S., Rao, Z., Qiu, Y., Starber, N., Mosber, B., & Radev, D. FOLIO: Natural language reasoning with first-order logic. *arXiv preprint arXiv:2209.00840*, 2022.

Jia, Y. and Harman, M. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let's verify step by step. *ICLR*, 2024.

Mellgren, J., Holmström, E., and Bjurling, M. Structural penalties for Prolog program synthesis via reinforcement learning. *arXiv preprint*, 2025.

Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Zhang, M., Li, Y., Wu, Y., and Guo, D. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Skalse, J., Howe, N., Krasheninnikov, D., and Krueger, D. Defining and characterizing reward hacking. *NeurIPS*, 2022.

Wang, X., Chen, L., and Liu, Y. EVOM: Cross-solver GRPO for formal verification. *arXiv preprint*, 2026.

Wu, Z., Bi, S., and Chen, Z. Agentic reasoning with logic programming. *arXiv preprint*, 2025.

Xie, Y., Li, Z., and Wang, H. Logic-RL: Reinforcement learning for formal logic reasoning. *arXiv preprint*, 2025.